System-level scheduling of rotating-chamber semiconductor cluster tools: A source-configurable two-stage MILP and structure-aware exact search

Lizhen Shao[a,*], Lincong Wei[a], Wei Wang[a]

[a]*Department of Control Science and Engineering, University of Science and Technology Beijing, No. 30 Xueyuan Road, Haidian District, Beijing 100083, China*

Abstract

Rotary-chamber cluster tools must coordinate product wafers, reusable dummy wafers, cleaning, robots, and chamber timing under full occupancy. We develop a source-configurable two-stage MILP that uses one internal model for a single-source specialization and dual-source mixed flow and covers both 4×1 parallel and 2×2 chained processing. Stage one minimizes end-to-end makespan; stage two preserves the discrete schedule and current makespan while improving waits and chamber cadence. Finite-time exact search is enhanced by a structure-preserving binary-skeleton (SPBS) initial solution and the role-aware hierarchical cut selector RDMCT-HBS. Exact equal-parameter comparisons show that pure 4×1 has the shortest makespan for recipe-unconstrained 8- and 16-wafer lots. Under the same 600-s budget, the two-stage workflow reduces four waiting and cadence measures by 39.70%–94.05%. SPBS raises incumbent availability from 51.67% to 100% and lowers mean PDI by 26.19%. Among seven nominal configurations sharing SPBS, RDMCT-HBS records the lowest observed mean PDI and solution time. The framework provides reproducible system-level scheduling while retaining solver-checked feasibility, bounds, and optimality status.

Keywords: semiconductor manufacturing, cluster tool, scheduling, mixed-integer linear programming, branch-and-cut, warm start

1. Introduction

Semiconductor manufacturing depends on capital-intensive, highly automated process equipment. Equipment utilization and production cadence depend not only on chamber processing times but also on coordination among wafer transport, load-lock switching, chamber occupancy, and control logic. A cluster tool (CT) integrates process modules, load locks, and wafer-handling robots in one controlled environment; residency limits, chamber cleaning, and activity-time variation further shrink the feasible scheduling space (Pan et al., 2018). CT scheduling is therefore an equipment-level manufacturing-system coordination problem rather than a generic parallel-machine problem.

Photolithography, etching, and deposition are three central stages of semiconductor manufacturing. In thin-film deposition, plasma-enhanced chemical vapor deposition (PECVD) for advanced processes is a major focus of equipment development. As throughput and floor-space requirements rise, conventional CTs no longer meet the full complexity of PECVD operation. Manufacturers therefore place several processing spaces inside a module and use chamber rotation to exchange the accessible positions, allowing several wafers to be processed while retaining robotic access to every station (Gu et al., 2024).

Carbon-containing films are difficult to clean, and the multi-space rotary chamber considered here must operate without empty stations. Reusable dummy wafers protect otherwise vacant positions. Product wafers enter from the atmospheric side, whereas a reusable-dummy storage module is mounted directly in the vacuum domain; the two flows merge at the chamber and compete for the same robot. Full occupancy and dummy filling create two representative modes: 4×1 processes four positions in parallel for one operation, whereas 2×2 processes two wafer pairs through a two-operation chain. Figure 1 shows the topology. If the recipe does not prescribe a mode, pure 4×1, pure 2×2, and mixed operation must be compared at the same wafer count and time parameters to separate intrinsic mode efficiency from mixed-flow coordination cost.

Existing exact scheduling studies cover complementary equipment boundaries. Qiao et al. (2022) represent cleaning occupancy with virtual wafers and optimize a dual-arm CT using binary integer programming. Li et al. (2022) link cleaning triggers to chamber state and derive feasible schedules for time-constrained single-arm tools. Gu et al. (2024) use linear programming to obtain minimum steady-state cycles for a four-finger robot and four-space modules. Yang et al. (2025) use MIP to determine release and robot-task sequences jointly for two wafer types. None of these models simultaneously represents finite-batch dual-source flow, reusable dummies, and both rotary recipes.

Learning-based CT controllers operate at a different decision layer. Kim and Lee (2025) combine deep reinforcement learning with look-ahead search to select non-cyclic actions, while Hong et al. (2026) train DDPG in a discrete-event model to adjust PVD process times. Here, processing times are inputs and the output must be a complete schedule with MILP feasibility checks, bounds, and an optimality status. Learning-enhanced exact optimization therefore provides a more suitable

*Corresponding author.

Email addresses: lshao@ustb.edu.cn (Lizhen Shao), m202420836@xs.ustb.edu.cn (Lincong Wei)

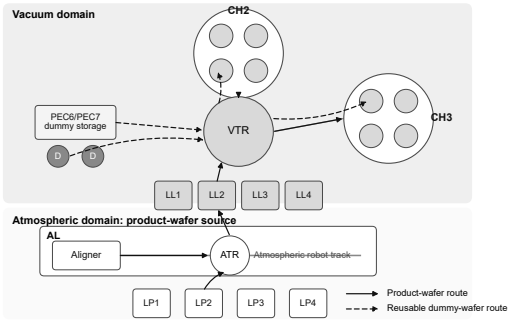


Figure 1: Dual-source mixed-flow cluster tool. Product wafers enter from the atmospheric domain and reusable dummy wafers from vacuum storage; both flows share the VTR and rotary chambers.

interface: Tang et al. (2020) formulate cut selection as reinforcement learning, Huang et al. (2022) learn candidate-cut rankings, Paulus et al. (2022) imitate a strong look-ahead rule, and Wang et al. (2023b) use a hierarchical policy to select both the cut budget and an ordered cut subset.

Two gaps follow. At the equipment level, no finite-horizon model unifies external source topology, full-occupancy recipes, reusable dummies, cleaning, and shared transport. At the solver level, generic cut features do not distinguish discrete equipment choices, continuous event times, and their coupling. This paper carries equipment structure through formulation, initialization, and cut selection. Its contributions are:

1. a source-configurable two-stage MILP that expresses a single-source specialization and dual-source mixed flow with the same internal model, covering $4{\times}1/2{\times}2$ processing, dummy reuse, cleaning, residency, and shared resources;

2. a throughput-first, operational-quality-second mechanism that reduces waits and cadence variation while preserving the discrete schedule and current makespan; and

3. an SPBS initial solution and the RDMCT-HBS cut selector, which improve finite-time primal and dual search without changing the MILP feasible region or proof logic.

2. Industrial problem and two-stage MILP

2.1. Model boundary, material routes, and notation

We formulate a finite-batch offline schedule. Inputs are the number and mode of product wafers, equipment action times, cleaning interval, residency limits, load-lock capacity, and dummy inventory. Outputs are complete routes for every product and dummy wafer, start and end times for each action, and the order of actions competing for a physical resource. Times are known within one solve and actions are non-preemptive. No buffer is introduced beyond the load ports, aligner, load locks, chambers, and dummy storage.

Product wafers are first paired by mode and index; equivalent pairs are then mapped by preprocessing to canonical compatible chamber positions. This removes only label-swapping symmetry. The MILP still decides active batches or chain positions, physical dummy identities, paired transfers, resource order, and all event times. Legal cleaning segments are likewise generated from the cleaning interval and complete recipe boundaries; the MILP activates the corresponding cleaning tasks and schedules their times rather than inserting cleaning at arbitrary instants.

A product follows LP–ATR–AL–upper load lock–VTR–rotary chamber–VTR–lower load lock–ATR–LP. A dummy travels from vacuum storage through the VTR to the chamber and then returns. A 4×1 batch loads two wafer pairs, processes all four positions synchronously, and unloads in last-in–first-out order; only the last incomplete batch may be filled by dummies. A 2×2 chain shares a product pair between adjacent cycles and follows dummy+P1, P1+P2, P2+P3, and P3+dummy. Figures 2 and 3 show the two event sequences.

Tables 1 and 2 list every symbol in the core formulation and distinguish sets/indices, input constants, fixed binary aliases, binary decisions, continuous decisions, and derived quantities. In the implementation, x^F and x^L are continuous $[0, 1]$ aliases fixed to zero or one by equality; this preserves uniform export names without creating extra branching decisions. To fit the official double-column template, the complete notation is divided into two full-width tables without deleting entries.

2.2. Source interfaces, operation activation, and time relations

The fixed incidence parameter satisfies

$$\sum_{s\in\mathcal{S}} \chi_{is} = 1, \qquad i \in \mathcal{I}. \tag{1}$$

Equation (1) assigns each material unit to exactly one source. Setting $|\mathcal{S}| = 1$ gives the single-source tool; using two interfaces and assigning products and dummies to different interfaces gives the dual-source tool.

If $\mathcal{A}^{\text{int}}$ is the internal network and $\mathcal{A}_s$ contains release and return arcs for source s, then

$$\mathcal{A} = \mathcal{A}^{\text{int}} \cup \left(\bigcup_{s\in\mathcal{S}} \mathcal{A}_s \right). \tag{2}$$

Equation (2) means that one- and two-source tools differ only at route endpoints; processing, cleaning, shared resources, and timing use the same internal model, so a dual-source tool is not the sum of two independent single-source schedules.

Canonical pair positions satisfy

$$\begin{aligned} x^F_{pcbl} &= \bar{x}^F_{pcbl}, \quad \sum_{c,b,l} x^F_{pcbl} = 1 \quad (p \in \mathcal{P}^F), \\ x^L_{pck} &= \bar{x}^L_{pck}, \quad \sum_{c,k} x^L_{pck} = 1 \quad (p \in \mathcal{P}^L). \end{aligned} \tag{3}$$

Equation (3) assigns each wafer pair to one compatible position and fixes equivalent locations to canonical indices. It removes label symmetry, not event times or resource order.

A position containing a pair must be active, and candidate positions are used consecutively:

$$\begin{aligned} x^F_{pcbl} &\le u^F_{cb}, \quad u^F_{c,b+1} \le u^F_{cb}, \\ x^L_{pck} &\le u^L_{ck}, \quad u^L_{c,k+1} \le u^L_{ck}. \end{aligned} \tag{4}$$

Table 1: Core notation I: sets, indices, and input parameters

Symbol	Domain and role	Meaning
$\mathcal{S}, \mathcal{I}$	sets/indices	Material-source interfaces and material units; $i \in \mathcal{I}$ may be a product or dummy demand.
$\mathcal{W}$	set/index	Product wafers, indexed by w.
$\mathcal{P}^F, \mathcal{P}^L$	sets/indices	Product pairs assigned to 4×1 and 2×2, indexed by p.
$\mathcal{C}$	set/index	Rotary process chambers, indexed by c.
$\mathcal{B}_c, \mathcal{K}_c$	ordered sets/indices	Candidate 4×1 batches b and 2×2 product-chain positions k in chamber c.
$\mathcal{L} = \{1, 2\}$	set/index	Front and back sides of a 4×1 batch.
$\mathcal{Q}_c, \mathcal{E}_c$	ordered sets/indices	2×2 process cycles q and cleaning segments e in chamber c.
$\mathcal{V}_c$	set/index	Union of all 4×1 batches and 2×2 cycles in chamber c.
$\mathcal{O}, \mathcal{A}$	sets/indices	Candidate physical operations and required process-precedence arcs.
$\mathcal{R}, \Lambda_r$	sets/indices	Shared resources and physical slots of resource r.
$\mathcal{J}_c^D, \mathcal{T}_c$	sets/indices	Dummy-wafer demands and reusable physical dummies of chamber c.
χ_{is}	known binary constant	Whether material unit i belongs to source interface s.
$\bar{x}^F_{pcbl}, \bar{x}^L_{pck}$	known binary constants	Canonical pair-to-4×1-side or pair-to-2×2-position assignment.
$\bar{z}_{ce}, \bar{a}^F_{cbe}, \bar{a}^L_{cqe}$	known binary constants	Active cleaning segments and pre-generated membership of batches or cycles.
$H, \bar{Q}_c$	nonnegative integer constants	Maximum cycles between cleanings and generated 2×2 cycles in chamber c.
$d_o, \delta_{oo'}$	nonnegative constants	Duration of operation o and minimum lag on arc (o, o').
d_y^F, d^L, r_c	nonnegative constants	4×1 stage duration, 2×2 process time, and chamber half-rotation time.
$\rho^r_{oo'}$	nonnegative constant	Empty travel or reset time from operation o to o' on resource r.
$M, \overline{R}_{wr}$	positive constants	Valid horizon-based big-M and maximum residency of wafer w in restricted region r.
κ_o, Γ_o	integer/set constants	Slots required by operation o and the slots it may use.
$\lambda_1, \ldots, \lambda_4$	nonnegative constants	Stage-two weights for total wait, maximum wait, cadence deviation, and soft-limit excess.
$\bar{\omega}_\alpha, \varepsilon_C$	nonnegative constants	Soft limit for wait α and the stage-two makespan tolerance.
$\mathcal{F}, \mathcal{D}_{\text{fix}}$	sets	Full MILP feasible region and discrete indices fixed in stage two.
$\mathcal{Q}^w, \mathcal{I}_h$	sets/indices	Wait quantities and process starts admitting cadence second differences.

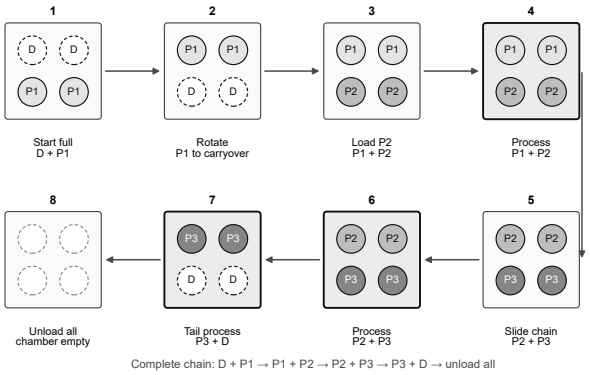


Figure 2: 2×2 chained processing. Dummies complete the chain head and tail, product pairs carry over between adjacent cycles, and all products and dummies are removed after the tail cycle.

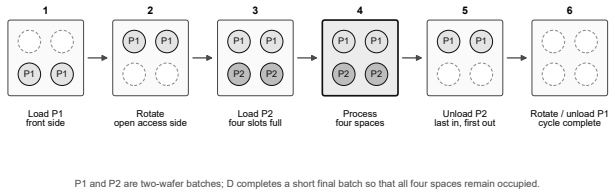


Figure 3: 4×1 parallel processing. Two wafer pairs occupy the two chamber sides, four positions process synchronously at full occupancy, and unloading follows last-in–first-out order.

Equation (4) activates any batch or chain position containing a product and excludes a nonphysical inactive position followed by an active one.

For each candidate operation o, activation and duration satisfy

$$0 \le s_o \le M b_o, \qquad 0 \le e_o \le M b_o, \qquad e_o = s_o + d_o b_o. \quad (5)$$

Equation (5) sets inactive operation times to zero and gives an active operation duration d_o. It applies to robot moves, alignment, pump/vent, rotation, product processing, dummy transport, and cleaning.

For every required process arc $(o, o') \in \mathcal{A}$,

$$b_{o'} \le b_o, \qquad s_{o'} \ge e_o + \delta_{oo'} - M(2 - b_o - b_{o'}). \quad (6)$$

Equation (6) activates a successor only with its predecessor and, when both exist, starts it after the predecessor and minimum lag. These arcs connect product routes, dummy routes, and load–process–unload chains.

2.3. Occupancy and event order in the two processing modes

For 4×1, let $A^F_{cb} = \sum_{p,l} x^F_{pcbl}$ be the number of product pairs assigned to batch (c, b). Full occupancy and dummy filling are

$$\begin{aligned} A^F_{cb} &= \sum_{p \in \mathcal{P}^F} \sum_{l \in \mathcal{L}} x^F_{pcbl}, \\ \sum_{p \in \mathcal{P}^F} x^F_{pcbl} + f^F_{cbl} &= u^F_{cb} \qquad (l \in \mathcal{L}), \\ \sum_{l \in \mathcal{L}} f^F_{cbl} &= 2u^F_{cb} - A^F_{cb}. \end{aligned} \quad (7)$$

Equation (7) gives each active 4×1 batch two sides, each occupied by one product pair or one dummy pair; all four chamber positions are therefore filled at process start.

Active batches are consecutive and dummy filling is restricted to the last active batch:

$$\begin{aligned} A^F_{cb} &\ge u^F_{cb}, & u^F_{c,b+1} &\le u^F_{cb}, \\ A^F_{cb} &\ge 2u^F_{c,b+1}, & t^F_{cb} &\le u^F_{cb}, \\ t^F_{cb} &\le 1 - u^F_{c,b+1}, & t^F_{cb} &\ge u^F_{cb} - u^F_{c,b+1}. \end{aligned} \quad (8)$$

Equation (8) requires at least one product pair in an active batch and two product pairs if a later batch is active. The last three inequalities set $t^F_{cb} = 1$ exactly at the active tail; the final candidate uses $t^F_{cB} = u^F_{cB}$.

Table 2: Core notation II: decision variables, fixed aliases, and derived quantities

Symbol	Domain and role	Meaning
x^F_{pcbl}, x^L_{pck}	fixed continuous $[0,1]$ aliases	Pair placement in a 4×1 batch side or a 2×2 chain position.
$u^F_{cb}, f^F_{cbl}, t^F_{cb}$	binary variables	4×1 batch activation, dummy filling on a side, and final active batch.
$u^L_{ck}, a^L_c, v^L_{cq}, h^L_{ck}$	binary variables	2×2 position, chain, process-cycle, and chain-tail activation.
$z_{ce}, a^F_{cbe}, a^L_{cqe}, q^{\mathrm{cl}}_{ch}$	fixed-binary aliases/variables	Cleaning segments, task membership, and cleaning-task activation.
$q_j, g_{j\tau}$	binary variables	Dummy demand and assignment to physical dummy τ.
$b_o, y^r_{oo'}$	binary variables	Operation activation and whether o precedes o' on resource r.
$h_{o\ell}$	binary slot variable	Whether operation o occupies physical slot ℓ.
γ_{pa}	binary variable	Whether pair p uses one paired transfer on robot route a.
s_o, e_o	nonnegative variables	Start and end times of operation o.
$s^{F,\nu}, e^{F,\nu}, s^L, e^L$	continuous aliases	Stage-specific notation for s_o, e_o in the two modes.
A^F_{cb}, N_{ce}	derived continuous/integer quantities	Product-pair count in a 4×1 batch and active cycles in a cleaning segment.
a_{vce}	fixed binary alias	Whether task $v\in\mathcal{V}_c$ belongs to cleaning segment e.
$C_w, C_{\max}$	nonnegative variables	Product completion time and lot makespan.
$\omega_\alpha, L_{\mathrm{sum}}, L_{\max}$	nonnegative variables/derived quantities	Stage-two wait or idle quantities, their sum, and their maximum.
$\Delta_{ceq}, v_{ceq}, \eta_\alpha$	derived/nonnegative variables	Cadence second difference, its absolute-value bound, and soft wait excess.
h_{ceq}, Φ	continuous alias/derived objective	Start of the qth process in a segment and stage-two schedule-quality objective.
$\xi, \widehat{\xi}, \widehat{C}$	vector/stage constants	Current variables, stage-one incumbent, and incumbent makespan.

Let FL, BL, P, BU, FU denote front load, back load, processing, back unload, and front unload. Then

$$\begin{aligned}
e^{F,\nu}_{cb} &= s^{F,\nu}_{cb} + d^F_\nu u^F_{cb} && (\nu\in\{FL,BL,P,BU,FU\}),\\
s^{F,BL}_{cb} &\ge e^{F,FL}_{cb} + r_c - M(1-u^F_{cb}),\\
s^{F,P}_{cb} &= e^{F,BL}_{cb},\\
s^{F,BU}_{cb} &\ge e^{F,P}_{cb} - M(1-u^F_{cb}),\\
s^{F,FU}_{cb} &\ge e^{F,BU}_{cb} + r_c - M(1-u^F_{cb}).
\end{aligned} \tag{9}$$

Equation (9) loads the front side, rotates to load the back, starts synchronous processing after the back load, and then unloads the back side before rotating and unloading the front. Consecutive batches are linked by $s^{F,FL}_{c,b+1} \ge e^{F,FU}_{cb} - M(1-u^F_{c,b+1})$.

For 2×2, each product pair occupies one ordered chain position:

$$u^L_{ck} = \sum_{p\in\mathcal{P}^L} x^L_{pck}, \qquad u^L_{c,k+1} \le u^L_{ck}, \qquad a^L_c = u^L_{c1}. \tag{10}$$

Equation (10) creates a one-to-one relation between product pairs and consecutive chain positions and uses a^L_c to indicate whether chamber c contains a 2×2 chain.

The generated layout supplies the number $\bar{Q}_c$ of process cycles, and h^L_{ck} identifies the chain tail:

$$\begin{aligned}
v^L_{cq} &= \begin{cases} a^L_c, & q\le\bar{Q}_c,\\ 0, & q>\bar{Q}_c,\end{cases}\\
\sum_k h^L_{ck} &= a^L_c, \qquad h^L_{ck}\le u^L_{ck}, \qquad h^L_{ck}+u^L_{c,k+1}\le 1,\\
h^L_{ck} &\ge u^L_{ck} - u^L_{c,k+1}.
\end{aligned} \tag{11}$$

Equation (11) gives $n+1$ process cycles for a segment with n product pairs: the head and tail contain one dummy pair and internal cycles share adjacent product pairs. The final relations make h^L_{ck} one only at the last active product position.

For a legal chain segment $(q_0,\dots,q_T)$, let H_0, B_q, and T_0 denote head loading, internal exchange, and tail unloading. Then

$$\begin{aligned}
e^L_{cq} - s^L_{cq} &= d^L v^L_{cq},\\
s^L_{cq_0} &= e^{H_0}_c,\\
s^B_{cq} &\ge e^L_{c,q-1} - M(1-a^L_c), \qquad s^L_{cq} = e^B_{cq} \quad (q_0<q\le q_T),\\
s^{T_0}_c &\ge e^L_{cq_T} - M(1-a^L_c).
\end{aligned} \tag{12}$$

Equation (12) begins the first process immediately after head loading, performs an internal exchange only after the previous cycle, and unloads the tail only after the last process. It directly realizes dummy+P1, P1+P2, …, Pn+dummy.

2.4. Cleaning and reusable dummy wafers

Cleaning segments are generated from H and complete process boundaries and enter the MILP as fixed binary aliases:

$$z_{ce} = \bar{z}_{ce}, \qquad a^F_{cbe} = \bar{a}^F_{cbe} u^F_{cb}, \qquad a^L_{cqe} = \bar{a}^L_{cqe} v^L_{cq}. \tag{13}$$

Equation (13) prevents arbitrary cleaning locations: 4×1 may split only at batch boundaries and 2×2 only after a complete chain segment, while the MILP still schedules process and cleaning times.

Let N_{ce} be the number of active process cycles in segment e:

$$\begin{aligned}
N_{ce} &= \sum_b a^F_{cbe} + \sum_q a^L_{cqe}, \qquad z_{ce}\le N_{ce}\le H z_{ce},\\
q^{\mathrm{cl}}_{ch} &= z_{c,h+1}.
\end{aligned} \tag{14}$$

Equation (14) places at least one and at most H process cycles in an active segment and activates cleaning between every pair of consecutive active segments.

Let $\mathcal{V}_c$ be the union of 4×1 batches and 2×2 cycles and let a_{vce} take the corresponding a^F or a^L. Cleaning order and the

physical cleaning chain satisfy

$$\begin{aligned} s_v &\ge e^{\mathrm{cl}}_{c,e-1} - M(1-a_{vce}) && (e>1),\\ e_v &\le s^{\mathrm{cl}}_{ce} + M(1-a_{vce}) + M(1-q^{\mathrm{cl}}_{ce}),\\ s^{\mathrm{cl,BL}}_{ce} &\ge e^{\mathrm{cl,FL}}_{ce} + r_c - M(1-q^{\mathrm{cl}}_{ce}),\\ s^{\mathrm{cl,P}}_{ce} &\ge e^{\mathrm{cl,BL}}_{ce} - M(1-q^{\mathrm{cl}}_{ce}),\\ s^{\mathrm{cl,BU}}_{ce} &\ge e^{\mathrm{cl,P}}_{ce} - M(1-q^{\mathrm{cl}}_{ce}),\\ s^{\mathrm{cl,FU}}_{ce} &\ge e^{\mathrm{cl,BU}}_{ce} + r_c - M(1-q^{\mathrm{cl}}_{ce}). \end{aligned} \tag{15}$$

Equation (15) starts a segment after the preceding cleaning and finishes it before the next cleaning. A cleaning task loads both chamber sides with dummies, rotates, processes, and unloads in reverse, thereby occupying the VTR, chamber, and four physical dummies rather than adding virtual downtime.

Dummy demand j is activated by a physical event:

$$q_j = \begin{cases} f^{F}_{cbl}, & j \text{ is a final-}4\times 1 \text{ filling demand},\\ a^{L}_{c}, & j \text{ is a } 2\times 2 \text{ head or tail demand},\\ q^{\mathrm{cl}}_{ch}, & j \text{ is a cleaning demand}. \end{cases} \tag{16}$$

Equation (16) prevents dummies from becoming an unlimited abstract filler: a physical demand exists only for a final batch, chain head/tail, or active cleaning task.

Each active demand selects one physical dummy from inventory:

$$\sum_{\tau\in\mathcal{T}_c} g_{j\tau} = q_j, \qquad j\in\mathcal{J}^{D}_{c}. \tag{17}$$

Equation (17) assigns every active demand exactly once. Mutual exclusion subsequently orders repeated uses of the same physical dummy, so it can be reused only after unloading and return to storage.

2.5. Robots, load locks, chambers, and time resources

ATR, AL, VTR, and each rotary chamber are unary resources; the upper and lower load locks each have two physical slots; each dummy in inventory is a reusable slot. An operation o requiring κ_o slots satisfies

$$\sum_{\ell\in\Gamma_o} h_{o\ell} = \kappa_o b_o. \tag{18}$$

Equation (18) assigns an active operation the required number of legal physical slots and assigns none to an inactive operation. AL, ATR, VTR, and a chamber have one slot, while load locks and dummy inventory retain physical identities.

If operations o and o' may compete for slot ℓ of resource r, then

$$\begin{aligned} s_{o'} &\ge e_o + \rho^{r}_{oo'} - M(1-y^{r}_{oo'}) - M(1-h_{o\ell}) - M(1-h_{o'\ell}),\\ s_{o} &\ge e_{o'} + \rho^{r}_{o'o} - My^{r}_{oo'} - M(1-h_{o\ell}) - M(1-h_{o'\ell}). \end{aligned} \tag{19}$$

Equation (19) makes $y^{r}_{oo'}$ choose one order whenever two actions use the same slot and includes empty robot travel or equipment reset through $\rho^{r}_{oo'}$. It applies to ATR, AL, both load locks, VTR, rotary chambers, and individual dummies, preventing concurrent robot actions and reuse before release.

An individual product or dummy stage must coincide with its selected batch, chain position, or cleaning window. For selection x and reference stage $[s^{\mathrm{ref}}, e^{\mathrm{ref}}]$,

$$\begin{aligned} -M(1-x) \le s_o - s^{\mathrm{ref}} \le M(1-x),\\ -M(1-x) \le e_o - e^{\mathrm{ref}} \le M(1-x). \end{aligned} \tag{20}$$

Equation (20) synchronizes the individual action with its physical stage when $x=1$ and releases the candidate relation when $x=0$, connecting pair positions, product routes, dummy demands, and chamber windows.

If pair p uses one paired transfer on robot route a, the two wafer stages are synchronized:

$$\begin{aligned} -M(1-\gamma_{pa}) \le s_{w_1 a} - s_{w_2 a} \le M(1-\gamma_{pa}),\\ -M(1-\gamma_{pa}) \le e_{w_1 a} - e_{w_2 a} \le M(1-\gamma_{pa}). \end{aligned} \tag{21}$$

Equation (21) means that $\gamma_{pa}=1$ moves a complete pair in one non-preemptive robot action. If $\gamma_{pa}=0$, the wafers still occupy the same robot separately and are ordered by Eq. (19).

For the entry and exit events of wafer w in restricted equipment region r,

$$0 \le s_{\mathrm{out}(w,r)} - e_{\mathrm{in}(w,r)} \le \overline{R}_{wr}. \tag{22}$$

Equation (22) limits the physical stay in an upper load lock, chamber, lower load lock, or restricted transport region. Load-lock occupation also spans placement, pump/vent, opposite-side pickup, and pressure reset, so placement by a robot is not mistaken for slot release.

2.6. Throughput-first and schedule-quality-second optimization

Let C_w be the time at which product wafer w completes the lower-load-lock return and reaches its load port. Stage one solves

$$C_w = e_{w,\mathrm{return}}, \qquad C_{\max} \ge C_w \ (w\in\mathcal{W}), \qquad \min_{\xi\in\mathcal{F}} C_{\max}. \tag{23}$$

Equation (23) measures makespan over the complete end-to-end route rather than at the last chamber operation. The feasible region $\mathcal{F}$ consists of Eqs. (1)–(22) expanded over all applicable indices.

Given stage-one incumbent $\widehat{\xi}$ and makespan $\widehat{C}$, stage two fixes the discrete structure indexed by $\mathcal{D}_{\mathrm{fix}}$ and minimizes linear schedule quality within the makespan tolerance:

$$\begin{aligned} \xi_d = \widehat{\xi}_d \quad (d\in\mathcal{D}_{\mathrm{fix}}), \qquad C_{\max} \le \widehat{C} + \varepsilon_C,\\ L_{\mathrm{sum}} = \sum_{\alpha\in\mathcal{Q}^w} \omega_\alpha, qquad L_{\max} \ge \omega_\alpha,\\ \Delta_{ceq} = (h_{ce,q+1} - h_{ceq}) - (h_{ceq} - h_{ce,q-1}),\\ v_{ceq} \ge \Delta_{ceq}, \qquad v_{ceq} \ge -\Delta_{ceq},\\ \eta_\alpha \ge \omega_\alpha - \bar{\omega}_\alpha, qquad \eta_\alpha \ge 0,\\ \min \Phi = \lambda_1 L_{\mathrm{sum}} + \lambda_2 L_{\max} + \lambda_3 \sum_{(c,e,q)\in\mathcal{I}_h} v_{ceq} + \lambda_4 \sum_{\alpha\in\mathcal{Q}^w} \eta_\alpha. \end{aligned} \tag{24}$$

Equation (24) preserves the stage-one makespan and principal discrete routes and resource orders while reducing total wait, maximum wait, interval variation within a cleaning segment, and optional soft-limit excess. The experiments use $(\lambda_1, \lambda_2, \lambda_3) = (1, 10, 25)$ and $\varepsilon_C = 10^{-6}$; the soft wait limit is disabled.

2.7. Coverage and model boundary

After index expansion, Eqs. (1)–(24) jointly describe the finite-batch dual-source problem with mixed $4 \times 1/2 \times 2$ operation, periodic cleaning, and finite dummy inventory. Equations (1)–(2) encode source interfaces: one interface is single source, while two interfaces with products and dummies assigned separately are dual source. Equations (3)–(6) encode pair locations, activation, durations, and precedence. Equations (7)–(9) encode 4×1 full occupancy, final-batch filling, rotation, and last-in–first-out unloading. Equations (10)–(12) encode the consecutive 2×2 chain, dummy-filled head/tail, and internal exchange. Equations (13)–(15) encode legal cleaning boundaries, segment capacity, and the physical cleaning chain. Equations (16)–(17) encode dummy demand and finite inventory. Equations (18)–(22) encode mutual exclusion of robots, load locks, chambers, and dummies, paired transfer, and residency. Equations (23)–(24) encode end-to-end makespan followed by wait/cadence refinement. A pure mode is obtained by making the other mode set empty.

Every active material route has a finite precedence chain from release to return, and every pair of operations competing for one resource must be ordered. If those inequalities formed a directed cycle, summing them would require an event to occur later than itself by a positive duration, so the combination is infeasible. The resource-ordering and process-precedence constraints therefore eliminate circular waits from the schedule.

The model boundary is explicit: preprocessing fixes wafer pairing, equivalent chamber sides, and legal cleaning segments; action times are deterministic within a solve; random failures and real-time rescheduling are excluded. These choices retain the central coupling among sources, modes, dummies, cleaning, and shared resources while removing label permutations with no physical distinction.

3. Structure-aware learning-enhanced exact solution

The primal difficulty is late arrival of a competitive feasible solution; the dual difficulty is that generic cut scores do not reveal whether a cut mainly tightens discrete choices, continuous event times, or their coupling. SPBS and RDMCT-HBS provide primal and dual guidance, respectively, while SCIP retains responsibility for feasibility checking, valid-cut generation, bounds, and optimality proofs (Achterberg, 2009; Bestuzheva et al., 2023).

3.1. SPBS initial solution

SPBS fixes deterministic product assignments, batch activation, cleaning segments, load locks, and part of the resource order in a temporary feasibility MIP while leaving event times free. If no incumbent is found, dummy, load-lock, and remaining order fixings are released in sequence. A candidate enters the production solve only after reconstruction and solver checking in the full MILP. It thus supplies a feasible starting point without excluding better solutions. In the benchmark, SPBS is precomputed offline and bound to the exact LP hash; its construction time is excluded from reported PDI and solution time.

3.2. Role features and hierarchical cut selection

Starting from the 13 generic HEM features, variables are partitioned by solver metadata into binary, general integer, objective-active continuous, continuous auxiliary, and other roles. The coefficient mass of role g in cut i is

$$p_{ig} = \frac{\sum_{j:x_j \in g} |a_{ij}|}{\sum_j |a_{ij}| + \varepsilon}. \tag{25}$$

Equation (25) is the fraction of absolute cut-coefficient mass carried by a mathematical variable role. Combined with bound, sign, coupling, and role-entropy terms, it gives 23 features without reading variable names or equipment labels.

A high-level policy selects the cut budget and a low-level pointer policy returns an ordered subset (Vinyals et al., 2015):

$$\pi_\theta(k, i_{1:k} \mid S) = \pi_\theta^H(k \mid S) \prod_{t=1}^{k} \pi_\theta^L(i_t \mid S, i_{1:t-1}). \tag{26}$$

Equation (26) jointly determines how many cuts to retain and in which order. Two workers collect independent SCIP trajectories, after which a central learner updates the shared policy synchronously. With $R = -\mathrm{PDI}$, the estimator is

$$\begin{aligned} \widehat{\nabla_\theta J} &= \frac{1}{B} \sum_{b=1}^{B} (R_b - \bar{R}_{\mathrm{EMA}}) \nabla_\theta \log \pi_\theta(\tau_b), \\ \bar{R}_{\mathrm{EMA}} &\leftarrow 0.9 \bar{R}_{\mathrm{EMA}} + 0.1 \bar{R}. \end{aligned} \tag{27}$$

Equation (27) is a batch policy-gradient estimator with an exponential-moving-average return baseline; B is the trajectory batch and τ_b is trajectory b. The baseline reduces return variation but is not a separately learned value function. The implementation is parallel hierarchical REINFORCE inspired by A3C, not an asynchronous actor–critic with a learned critic (Williams, 1992; Mnih et al., 2016).

Training uses five instances and five random seeds. Each seed uses two workers; every epoch collects eight 30-s/1024-MB trajectories, and training ends at epoch 60. Adam learning rates are 10^{-4} for the low-level policy and 5×10^{-4} for the high-level policy. Each method–seed combination uses the preregistered epoch-60 parameters without benchmark- or test-set selection. At inference, bounded beam search preserves multiple partial sequences (Choo et al., 2022), and role-aware submodular completion balances policy score, instantaneous cut quality, role coverage, and candidate-pool representation (Nemhauser et al., 1978). The approximation property applies only to the residual monotone completion problem after anchors and the candidate pool are fixed; it is not a global cut-selection guarantee.

RDMCT-HBS only reorders candidates already generated and validated by SCIP, and SPBS submits only solver-checked feasible solutions. Neither changes the objective, constraints, integrality, or bound logic; feasibility and optimality retain the same meaning as in default SCIP.

4. Computational results and analysis

4.1. Experimental design and metrics

Seven complementary evidence sets are used. Four small exact solves check internal implementation consistency. A 2700-run nominal study compares seven solver-guidance configurations on 20 manufacturing instances; its 150-run exact subset supplies the equal-wafer-count mode comparison. A 960-run matched study evaluates precomputed SPBS input, and 1500 runs compare three equal-total-budget workflows. A 300-run factorial design associates scale, recipe composition, and process duration with difficulty; 80 one-factor runs assess operation under parameter perturbations; and 729 distribution-shift runs test feasible-schedule delivery on 40–64-wafer batches. The six formal benchmark groups contain 6269 runs and the four exact checks bring the total to 6273. All 6041 saved incumbents pass SCIP and an independent post-solve check against the same serialized MILP; no invalid schedule is found.

The MILP, SCIP version, instances, and random seeds are frozen, and each comparison family uses common time and memory limits. The main metric is the normalized primal–dual gap integral over a common horizon $[0, T]$, $\mathrm{PDI} = \int_0^T g(t)\,\mathrm{d}t$; lower values mean that the bound gap closes earlier. PDI can be zero only if the gap is zero for almost the entire timed horizon. A zero terminal gap instead means that optimality is proved at termination and does not imply zero PDI. The manufacturing instance is the inferential unit: random repeats are first aggregated within an instance, after which paired instance differences use nonparametric 95% bootstrap intervals, Wilcoxon signed-rank tests, and Holm correction within each registered analysis family (Gleixner et al., 2021). Time-limited runs remain in the summaries so that random seeds are not treated as independent manufacturing evidence. Full hardware, budgets, tolerances, instance lists, and hyperparameters are provided in the supplementary reproducibility package.

4.2. Processing mode and two-stage operational value

Figure 4 compares pure 4×1, pure 2×2, and 1:1 mixed operation at 8 and 16 wafers under the same topology, transport/cleaning parameters, processing scale, and 600-s budget. All six configurations reach zero terminal gap; each result is confirmed by the complete cross of five solver and five training seeds.

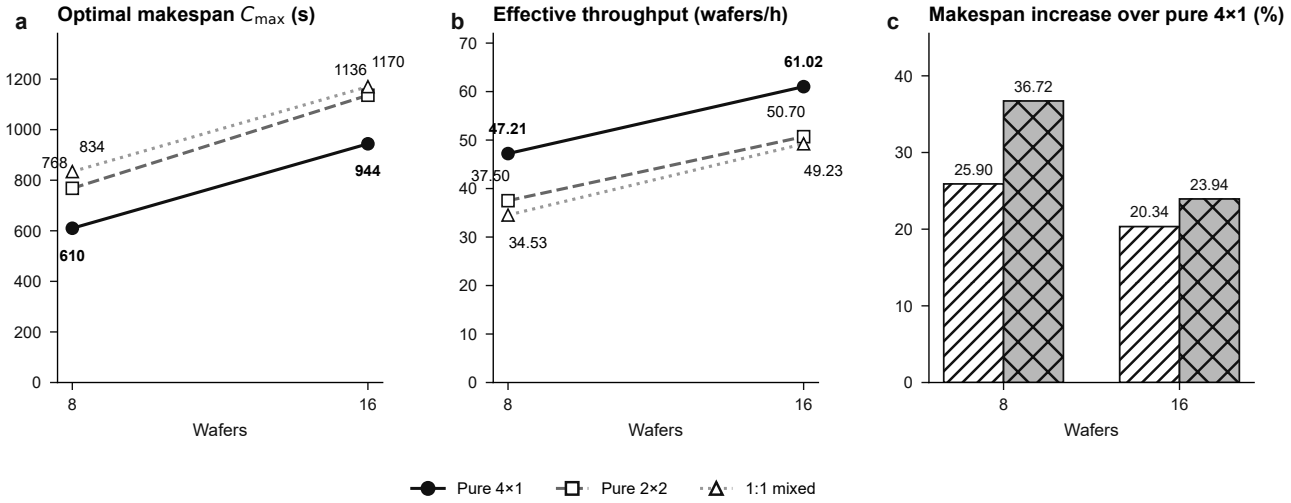


Figure 4: Mode efficiency at equal wafer counts. a, exact optimal C_{max}; b, equivalent throughput $3600n/C_{max}$; c, makespan increase of pure 2×2 and mixed operation over pure 4×1. Each point represents 25 exact repetitions, 150 runs in total; values are labeled directly.

At 8 wafers, the three makespans are 610, 768, and 834 s and the equivalent throughputs are 47.21, 37.50, and 34.53 wafers/h. At 16 wafers, the corresponding values are 944, 1136, and 1170 s and 61.02, 50.70, and 49.23 wafers/h. Thus, when the recipe does not prescribe a mode and exposure times are equal, pure 4×1 achieves both the shortest makespan and the highest equivalent throughput for the 8- and 16-wafer batches.

For the same 600-s total budget, the complete two-stage workflow assigns 540 s to the main solve and up to 60 s to refinement, whereas the one-stage control assigns all 600 s to the main solve. Figure 5 shows that the two-stage workflow reduces total route wait, maximum wait, cadence coefficient of variation, and cadence deviation by 51.91%, 68.89%, 39.70%, and 94.05%, respectively. All four paired instance differences remain significant after Holm correction, demonstrating that the two-stage workflow markedly improves waiting and cadence quality under the same total budget.

The quadratic-objective control and the complete linear refinement both use the 540+up-to-60-s split. Linear refinement yields a cadence deviation of 22.04, 63.38% below the quadratic profile's 60.18, and the matched difference remains significant after Holm correction ($p = 0.0377$); total route wait, maximum wait, and cadence coefficient of variation are similar overall. The linear objective therefore produces a more regular processing cadence under the same budget.

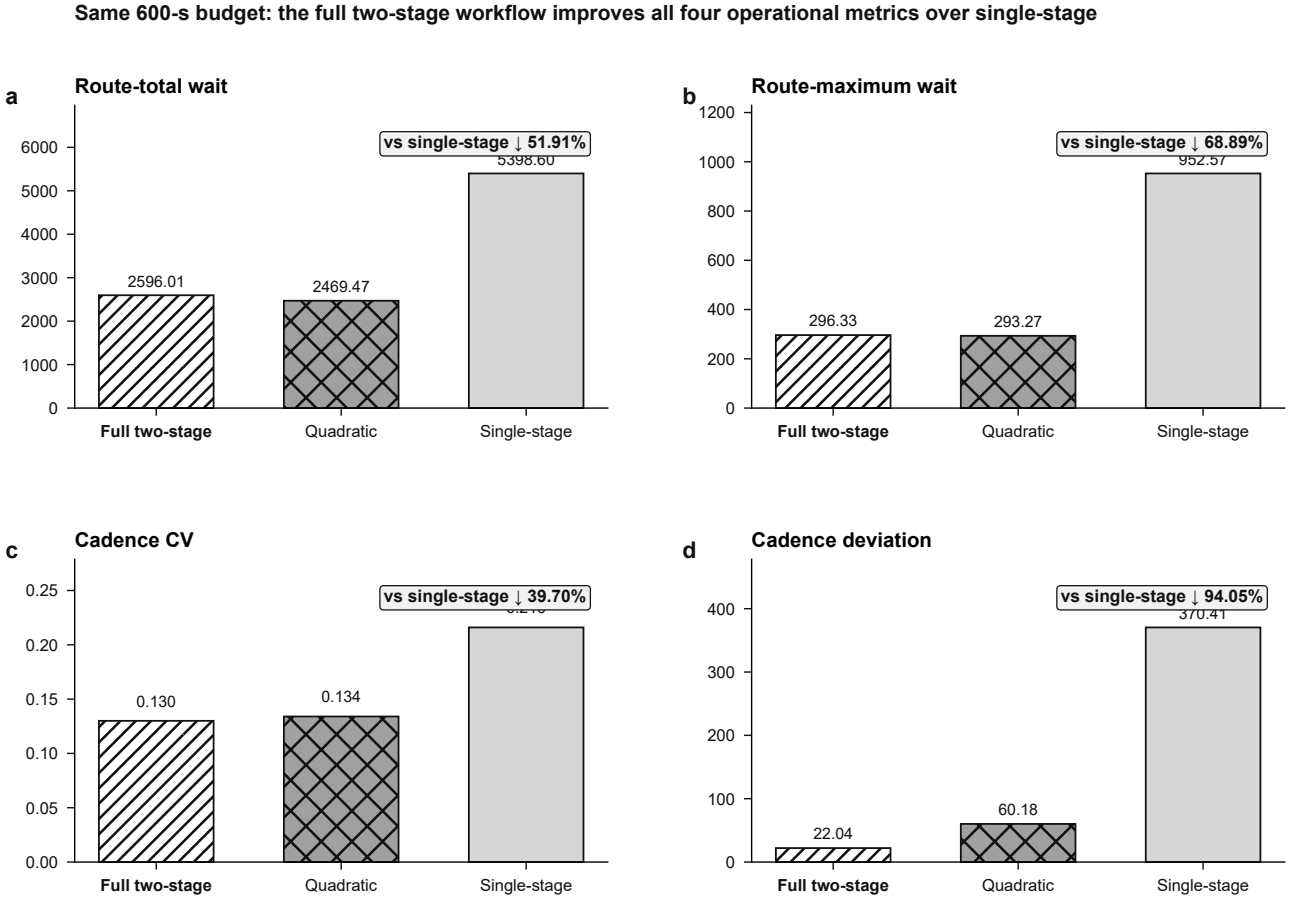


Figure 5: Final-schedule quality under the same 600-s total budget. a–d, total route wait, maximum route wait, cadence coefficient of variation, and cadence deviation; 1500 runs in total, with observed means labeled above the bars.

4.3. SPBS and structure-aware cut selection

The paired SPBS study uses 48 instances and ten paired seeds per instance. Relative to the no-start condition, precomputed SPBS increases incumbent availability from 51.67% to 100%, a gain of 48.33 percentage points (95% interval $[34.58, 62.29]$). Mean PDI falls from 27473.05 to 20278.78, a decrease of 7194.27 or 26.19% (95% interval $[5145.78, 9317.61]$). Both paired differences remain significant after Holm correction ($p = 2.55\times10^{-5}$ and $p = 3.46\times10^{-8}$). These results show that SPBS substantially increases finite-time access to complete feasible schedules and improves primal–dual search efficiency.

In 2700 nominal runs where all policies share the same SPBS incumbent, RDMCT-HBS obtains the lowest observed mean PDI (16585.70) and shortest mean solution time; all 500 of its runs return feasible solutions (Fig. 7). The largest mean improvement occurs when the 13 generic features are replaced by the 23 role-aware features. Role completion yields a further reduction, while beam decoding provides only a small improvement on top of the structure-aware representation. Wider decoding alone therefore does not explain the complete method's rank.

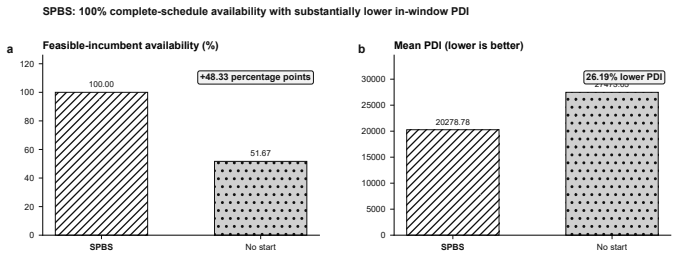


Figure 6: Effect of precomputed SPBS on primal search. a, incumbent availability; b, mean PDI. Each condition has $n = 480$; observed values are labeled directly.

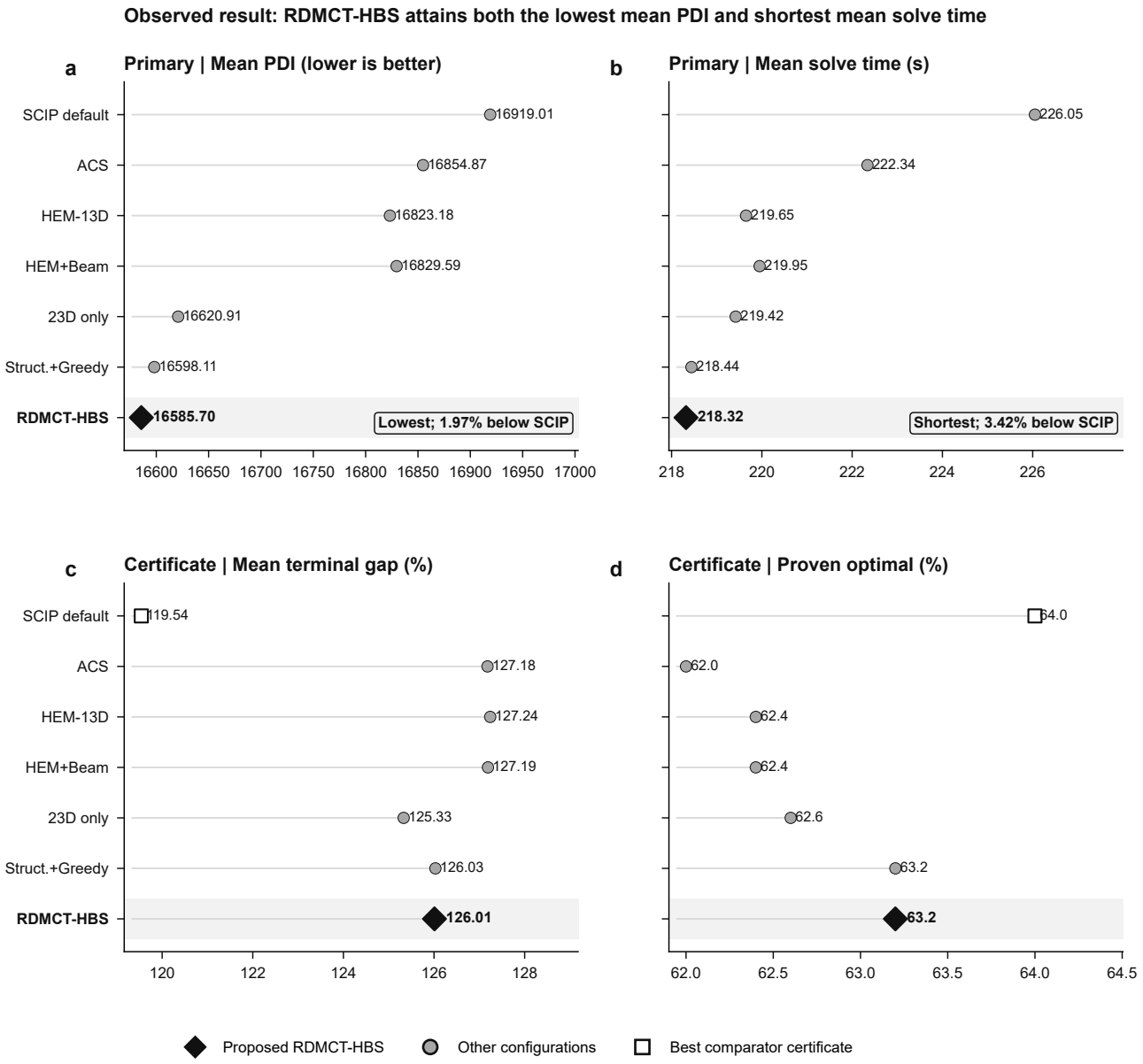


Figure 7: Nominal comparison of seven solver-guidance configurations sharing SPBS. Panels report mean PDI, mean solution time, mean terminal gap, and proven-optimal rate; 2700 runs in total, with values labeled beside points. The black diamond marks RDMCT-HBS; an open square marks a proof metric led by another configuration.

Across the four metrics, RDMCT-HBS achieves the lowest mean PDI and shortest mean solution time over the finite-time trajectory, while default SCIP is slightly better in terminal gap and proven-optimal rate. The result shows that combining role-aware features, completion, and beam decoding improves finite-time search trajectories.

4.4. Scale, recipe composition, and computational difficulty

The factorial design contains $4 \times 5 \times 3 = 60$ wafer-count–recipe-composition–process-scale cells with five solver seeds per cell, for 300 runs. Figure 8 shows a sharp transition. All 75 runs at both 8 and 16 wafers prove optimality, with mean PDI values of 28.13 and 1876.54 and mean solution times of 0.34 and 33.86 s. At 24 and 32 wafers, the proven-optimal rates fall to 36% and 20%, mean PDI rises to 31155.94 and 32959.89, and mean solution time reaches 408.55 and 433.20 s.

The prespecified PDI factorial model has $R^2 = 0.998$. Under the 1:1 mixed recipe, the additional interaction coefficients at 24 and 32 wafers are 37729.48 and 40788.63, while the 95% intervals for the process-scale main effect and all prespecified process-scale interactions include zero. Over the tested 0.9–1.1 process-scale range, computational difficulty is driven primarily by the interaction between batch size and mixed-flow composition, with a comparatively small process-scale effect.

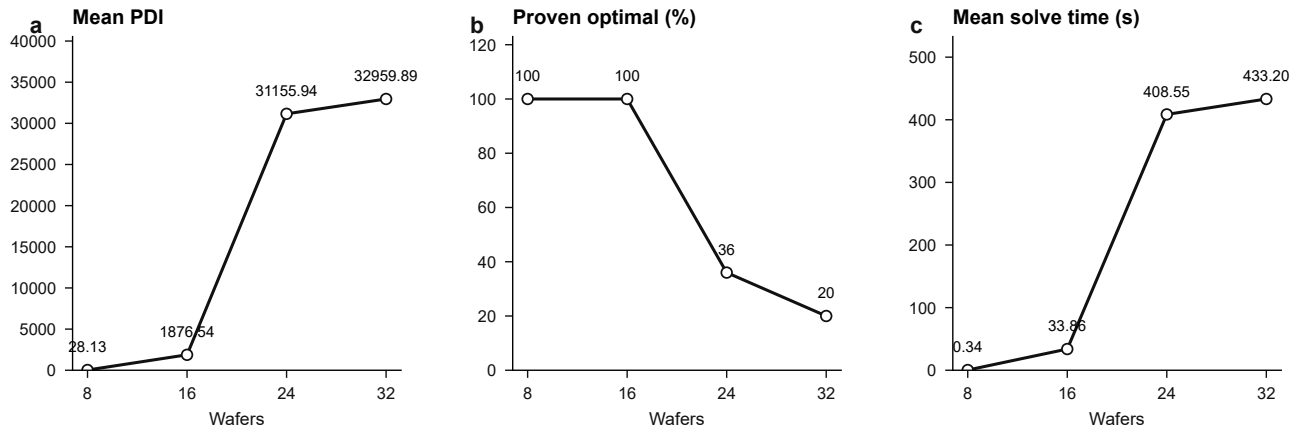


Figure 8: Scale transition in the 300-run factorial experiment. a, mean PDI; b, proven-optimal rate; c, mean solution time. Each wafer count contains 75 runs and observed values are labeled directly.

4.5. Operation under parameter perturbations

The one-factor sensitivity study covers eight prescribed perturbations of transport, processing, cleaning interval, and vacuum-domain dummy inventory, for 80 runs. Every run returns a solver-checked feasible schedule and 20 runs prove optimality; scenario mean PDI ranges from 28899.41 to 42478.32. A longer cleaning interval gives the lowest mean PDI and slower transport the highest, showing that cleaning frequency and shared-robot transport cadence materially shape search difficulty. The framework nevertheless returns a feasible schedule in every prescribed perturbation scenario.

4.6. Feasible-schedule delivery under distribution shift

The distribution-shift suite enlarges batches to 40–64 wafers and changes mixed-flow composition, producing 729 runs over nine configurations. All 729 runs return an incumbent that passes SCIP and independent post-solve checks, showing that the SPBS–MILP–SCIP solution pipeline reliably delivers verifiable schedules at the larger shifted scale.

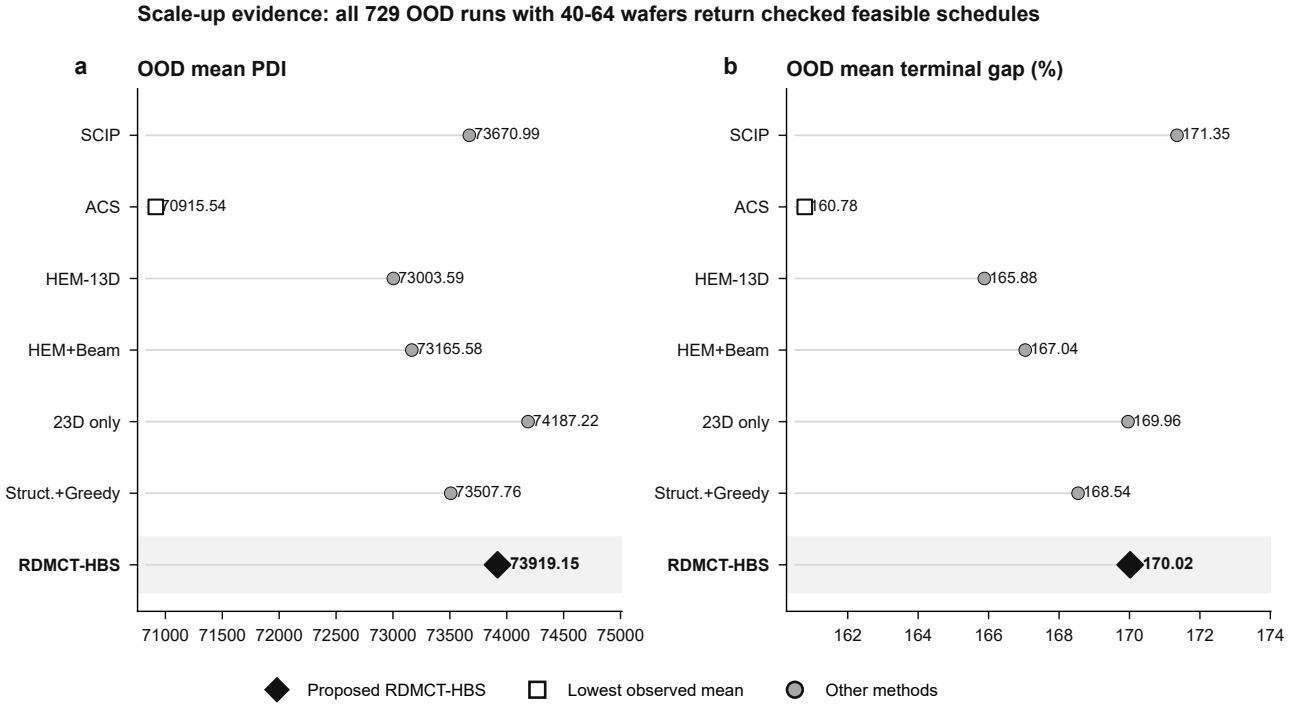


Figure 9: Distribution-shift experiment with 40–64 wafers. a, instance-first mean PDI for seven configurations; b, mean terminal gap. All 729 runs return checked feasible schedules; observed values are labeled directly.

In the distribution-shift experiment, adaptive cut selection records the lowest mean PDI (70915.54) and terminal gap (160.78%). The corresponding values are 73919.15 and 170.02% for RDMCT-HBS, and 73670.99 and 171.35% for default SCIP. Method rankings change on the larger instances, while the shared SPBS–MILP–SCIP pipeline consistently returns a checked feasible schedule in all 729 runs.

4.7. Industrial interpretation and integrated findings

The experiments support three industrial conclusions. When the recipe does not prescribe a mode, pure 4×1 provides the highest throughput for the 8- and 16-wafer batches. Under the same total budget, the two-stage workflow significantly improves waiting and cadence quality. SPBS and structure-aware cut selection improve the primal start and finite-time search trajectory, respectively. The factorial, sensitivity, and distribution-shift studies further identify batch size and mixed-flow composition as the main drivers of computational difficulty and show that the framework continues to deliver checked schedules under parameter perturbations and on 40–64-wafer instances.

5. Conclusions

This study develops a finite-horizon two-stage MILP for dual-source rotary-chamber semiconductor cluster tools that unifies 4×1 parallel processing, 2×2 chained processing, reusable dummy wafers, cleaning, and shared transport. The formulation is coupled with an SPBS initial solution and the structure-aware hierarchical cut selector RDMCT-HBS. Exact mode comparisons show that pure 4×1 achieves the shortest makespan and highest equivalent throughput for both 8- and 16-wafer batches. Under the same 600-s total budget, the two-stage workflow reduces total route wait, maximum route wait, cadence coefficient of variation, and cadence deviation by 51.91%, 68.89%, 39.70%, and 94.05%, respectively. SPBS raises finite-time feasible-solution availability from 51.67% to 100% and reduces mean PDI by 26.19%. In the seven-method nominal comparison with a common SPBS start, RDMCT-HBS achieves the lowest mean PDI and shortest mean solution time. The factorial experiment identifies the interaction between batch size and mixed-recipe composition as the main driver of computational difficulty, and all 729 distribution-shift runs on 40–64-wafer instances return solver-checked feasible schedules.

These results show that embedding equipment structure across MILP formulation, two-stage schedule-quality optimization, primal initialization, and cut selection improves both manufacturing schedule quality and finite-time exact-search efficiency. Future work will extend the source-interface model to stochastic arrivals, equipment disturbances, and online rescheduling, and will assess SPBS construction, rolling optimization, and line-level deployment jointly.

CRediT authorship contribution statement

Lizhen Shao and Lincong Wei jointly contributed to conceptualization, methodology, software, validation, formal analysis, investigation, visualization, and manuscript preparation. All authors reviewed and approved the manuscript.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data and code availability

The data, mathematical instances, trained models, and source code supporting the findings are available from the corresponding author on reasonable request. A permanent public archive will be cited in the version of record.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During manuscript preparation, the authors used OpenAI ChatGPT and Codex for language editing, LaTeX consistency checks, and software debugging. The authors reviewed and edited the output and take full responsibility for the content of the publication.

References

Pan, C.R., Zhou, M.C., Qiao, Y., Wu, N.Q., 2018. Scheduling cluster tools in semiconductor manufacturing: Recent advances and challenges. IEEE Transactions on Automation Science and Engineering 15, 2, 586–601. doi:10.1109/TASE.2016.2642997.

Gu, L., Wu, N., Qiao, Y., Zhang, S., Li, T., 2024. Scheduling cluster tools with multi-space process modules and a multi-finger-arm robot in wafer fabrication subject to wafer residency time constraints. Applied Sciences 14, 9490. doi:10.3390/app14209490.

Achterberg, T., 2009. SCIP: Solving constraint integer programs. Mathematical Programming Computation 1, 1–41. doi:10.1007/s12532-008-0001-1.

Bestuzheva, K., Besançon, M., Chen, W.K., Chmiela, A., Donkiewicz, T., van Doornmalen, J., Eifler, L., Gaul, O., Gamrath, G., Gleixner, A., Gottwald, L., Graczyk, C., Halbig, K., Hoen, A., Hojny, C., van der Hulst, R., Koch, T., Lübbecke, M.E., Maher, S.J., Matter, F., Mühmer, E., Müller, B., Pfetsch, M.E., Schlösser, F., Serrano, F., Shinano, Y., Sofranac, B., Turner, M., Vigerske, S., Wegscheider, F., Wellner, P., Weninger, D., Witzig, J., 2023. Enabling research through the SCIP optimization suite 8.0. ACM Transactions on Mathematical Software 49, 22:1–22:21. doi:10.1145/3585516.

Choo, J., Kwon, Y.D., Kim, J., Jae, J., Hottung, A., Tierney, K., Gwon, Y., 2022. Simulation-guided beam search for neural combinatorial optimization, in: Advances in Neural Information Processing Systems. URL: https://papers.nips.cc/paper_files/paper/2022/hash/39b9b60f0d149eabd1fff2d7c7d5afc4-Abstract-Conference.html, doi:10.52202/068431-0637.

Gleixner, A., Hendel, G., Gamrath, G., Achterberg, T., Bastubbe, M., Berthold, T., Christophel, P., Jarck, K., Koch, T., Linderoth, J., Lubbecke, M.E., Mittelmann, H.D., Ozyurt, D., Ralphs, T.K., Salvagnin, D., Shinano, Y., 2021. MIPLIB 2017: Data-driven compilation of the 6th mixed-integer programming library. Mathematical Programming Computation 13, 443–490. doi:10.1007/s12532-020-00194-3.

Gu, L., Wu, N., Qiao, Y., Zhang, S., 2024. Scheduling of Multi-finger-robotic Cluster Tools with Multi-space Process Modules, in: 2024 International Conference on Networking, Sensing and Control (ICNSC), IEEE. doi:10.1109/ICNSC62968.2024.10760109.

Hong, T.Y., Hsu, C.Y., Chen, C.L., 2026. Simulation-based deep reinforcement learning for process time optimization in semiconductor cluster tool and an empirical study. Computers & Industrial Engineering 211, 111698. doi:10.1016/j.cie.2025.111698.

Huang, Z., Wang, K., Liu, F., Zhen, H.L., Zhang, W., Yuan, M., Hao, J., Yu, Y., Wang, J., 2022. Learning to select cuts for efficient mixed-integer programming. Pattern Recognition 123, 108353. doi:10.1016/j.patcog.2021.108353.

Kim, H.J., Lee, J.H., 2025. Scheduling cluster tools for concurrent processing: Deep reinforcement learning with adaptive search. IEEE Transactions on Automation Science and Engineering 22, 3783–3796. doi:10.1109/TASE.2024.3399818.

Li, C., Yang, F., Zhen, L., 2022. Efficient scheduling approaches to time-constrained single-armed cluster tools with condition-based chamber cleaning operations. International Journal of Production Research 60, 3555–3568. doi:10.1080/00207543.2021.1926568.

Mnih, V., Badia, A.P., Mirza, M., Graves, A., Lillicrap, T.P., Harley, T., Silver, D., Kavukcuoglu, K., 2016. Asynchronous methods for deep reinforcement learning, in: Proceedings of the 33rd International Conference on Machine Learning, PMLR. pp. 1928–1937. URL: https://proceedings.mlr.press/v48/mniha16.html.

Nemhauser, G.L., Wolsey, L.A., Fisher, M.L., 1978. An analysis of approximations for maximizing submodular set functions—i. Mathematical Programming 14, 265–294. doi:10.1007/BF01588971.

Paulus, M.B., Zarpellon, G., Krause, A., Charlin, L., Maddison, C., 2022. Learning to cut by looking ahead: Cutting plane selection via imitation learning, in: Proceedings of the 39th International Conference on Machine Learning, PMLR. pp. 17584–17600. URL: https://proceedings.mlr.press/v162/paulus22a.html.

Qiao, Y., Lu, Y., Li, J., Zhang, S., Wu, N., Liu, B., 2022. An efficient binary integer programming model for residency

time-constrained cluster tools with chamber cleaning requirements. IEEE Transactions on Automation Science and Engineering 19, 1757–1771. doi:10.1109/TASE.2021.3122576.

Tang, Y., Agrawal, S., Faenza, Y., 2020. Reinforcement learning for integer programming: Learning to cut, in: Proceedings of the 37th International Conference on Machine Learning, PMLR. pp. 9367–9376. URL: https://proceedings.mlr.press/v119/tang20a.html.

Vinyals, O., Fortunato, M., Jaitly, N., 2015. Pointer networks, in: Advances in Neural Information Processing Systems, pp. 2692–2700.

Wang, Z., Li, X., Wang, J., Kuang, Y., Yuan, M., Zeng, J., Zhang, Y., Wu, F., 2023b. Learning cut selection for mixed-integer linear programming via hierarchical sequence model, in: Proceedings of the Eleventh International Conference on Learning Representations. URL: https://openreview.net/forum?id=Zob4P9bRNcK.

Williams, R.J., 1992. Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine Learning 8, 229–256. doi:10.1007/BF00992696.

Yang, F., Li, C., Zhen, L., Zhang, C., Wu, N., 2025. Scheduling residency time-constrained single-armed cluster tools with two-wafer types via mixed integer programming model. Expert Systems with Applications 288, 128209. doi:10.1016/j.eswa.2025.128209.